Proof of Concept (POC) Implementation Guide

Data Vault 2.0 Tax System Data Warehouse — Chapter 5 POC Demonstrations

Author: Sot So **Advisor:** Mr. Chap Chanpiseth **Institution:** Royal University of Phnom Penh — Master of Science in Data Science and Engineering **Date:** February 2026

Overview

This guide provides detailed, step-by-step instructions to reproduce all four Proof of Concept (POC) demonstrations presented in Chapter 5, Section 5.2 of the thesis. Each POC corresponds to a challenge identified in the Problem Statement (Section 1.2) and demonstrates how the implemented Data Vault 2.0 methodology addresses it.

Prerequisites: - SQL Server 2022+ (or SQL Server 2025) - SSIS 2022 (Visual Studio 2022 with SSIS extension) - All 6 databases created and populated via scripts 00–04 - ETL Control Framework deployed via script 03 - Full Load pipeline executed successfully via script 00 (or scripts 05–09 manually)

Database Architecture:

Database	Layer	Purpose
TaxSystemDB	Source	Simulated OLTP tax system (9 tables, 1,000 taxpayers)
DV_Staging	Staging	Truncate-and-reload landing zone (9 STG tables)
DV_Bronze	Bronze (Raw Vault)	Hub-Satellite-Link model (9 Hubs, 9 Satellites, 5 Links)
DV_Silver	Silver (Business Vault)	Derived metrics (3 PIT, 1 Bridge, 2 BUS tables)
DV_Gold	Gold (Star Schema)	BI-ready dimensions and facts (7 DIMs, 4 FACTs)
ETL_Control	Control	ETL orchestration (9 control tables)

POC 1: Schema Flexibility via Hub-Satellite Separation

Challenge: Traditional data warehouses require significant restructuring when source systems change — ALTER TABLE on dimensions, ETL pipeline modifications, index rebuilds, and downstream report updates.

Objective: Demonstrate that Data Vault 2.0 accommodates new attributes by creating a new Satellite table with zero impact on existing objects.

Step 1: Verify Current Bronze Layer Structure

Before making any changes, confirm the existing objects related to the Taxpayer entity.

```
-- Connect to: DV_Bronze
USE DV_Bronze;
GO

-- Step 1a: Verify HUB_Taxpayer structure
SELECT
    c.COLUMN_NAME,
    c.DATA_TYPE,
    c.CHARACTER_MAXIMUM_LENGTH,
    c.IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS c
WHERE c.TABLE_NAME = 'HUB_Taxpayer'
ORDER BY c.ORDINAL_POSITION;
```

Expected Result:

COLUMN_NAME	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	IS_NULLABLE
HUB_Taxpayer_HK	varbinary	32	NO
TaxID	varchar	20	NO
HUB_LoadDate	datetime	NULL	NO
HUB_RecordSource	nvarchar	50	NO

```
-- Step 1b: Verify SAT_Taxpayer structure (current descriptive attributes)
SELECT
    c.COLUMN_NAME,
    c.DATA_TYPE,
    c.CHARACTER_MAXIMUM_LENGTH,
    c.IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS c
WHERE c.TABLE_NAME = 'SAT_Taxpayer'
ORDER BY c.ORDINAL_POSITION;
```

Expected Result:

COLUMN_NAME	DATA_TYPE	LENGTH	IS_NULLABLE
HUB_Taxpayer_HK	varbinary	32	NO
SAT_LoadDate	datetime	NULL	NO
SAT_EndDate	datetime	NULL	YES
SAT_HashDiff	varbinary	32	NO
SAT_RecordSource	nvarchar	50	NO
LegalBusinessName	varchar	300	YES
TradingName	varchar	300	YES
CategoryID	int	NULL	YES

COLUMN_NAME	DATA_TYPE	LENGTH	IS_NULLABLE
StructureID	int	NULL	YES
RegistrationDate	date	NULL	YES
EstimatedAnnualRevenue	decimal	NULL	YES
IsActive	bit	NULL	YES

```sql
-- Step 1c: Count all existing Taxpayer-related objects
SELECT 'HUB_Taxpayer' AS ObjectName, 'Hub' AS ObjectType, COUNT(*) AS
RowCount
FROM DV_Bronze.dbo.HUB_Taxpayer
UNION ALL
SELECT 'SAT_Taxpayer', 'Satellite', COUNT(*) FROM DV_Bronze.dbo.SAT_Taxpayer
UNION ALL
SELECT 'LNK_TaxpayerDeclaration', 'Link', COUNT(*) FROM
DV_Bronze.dbo.LNK_TaxpayerDeclaration
UNION ALL
SELECT 'LNK_TaxpayerOfficer', 'Link', COUNT(*) FROM
DV_Bronze.dbo.LNK_TaxpayerOfficer
UNION ALL
SELECT 'LNK_TaxpayerOwner', 'Link', COUNT(*) FROM
DV_Bronze.dbo.LNK_TaxpayerOwner;
```

Step 2: Create New Satellite (Only Action Required)

A new requirement arrives: "Track TaxpayerEmail and TaxpayerPhone for each taxpayer." Under Data Vault 2.0, this requires only creating a new Satellite — no existing objects are touched.

```sql
-- Connect to: DV_Bronze
USE DV_Bronze;
GO

-- Step 2a: CREATE the new Satellite table
CREATE TABLE SAT_Taxpayer_Contact (
    HUB_Taxpayer_HK     VARBINARY(32)   NOT NULL,
    SAT_LoadDate         DATETIME        NOT NULL DEFAULT GETDATE(),
    SAT_EndDate          DATETIME        NULL,
    SAT_RecordSource     NVARCHAR(50)    NOT NULL,
    SAT_HashDiff         VARBINARY(32)   NOT NULL,
    TaxpayerEmail        VARCHAR(200)    NULL,
    TaxpayerPhone        VARCHAR(50)     NULL,
    CONSTRAINT PK_SAT_Taxpayer_Contact
        PRIMARY KEY (HUB_Taxpayer_HK, SAT_LoadDate),
    CONSTRAINT FK_SAT_TpContact_HUB
        FOREIGN KEY (HUB_Taxpayer_HK)
        REFERENCES HUB_Taxpayer (HUB_Taxpayer_HK)
);
GO
```

```sql
-- Step 2b: Create filtered index for current records (performance
optimization)
CREATE INDEX IX_SAT_Taxpayer_Contact_Current
    ON SAT_Taxpayer_Contact(HUB_Taxpayer_HK)
    WHERE SAT_EndDate IS NULL;
GO

PRINT '✓ SAT_Taxpayer_Contact created successfully';
```

Step 3: Verify Zero Impact on Existing Objects

```sql
-- Step 3a: Confirm HUB_Taxpayer is unchanged (exact same columns, same row
count)
SELECT
    c.COLUMN_NAME, c.DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS c
WHERE c.TABLE_NAME = 'HUB_Taxpayer'
ORDER BY c.ORDINAL_POSITION;
-- Expected: Identical to Step 1a — 4 columns, no changes

-- Step 3b: Confirm SAT_Taxpayer is unchanged
SELECT
    c.COLUMN_NAME, c.DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS c
WHERE c.TABLE_NAME = 'SAT_Taxpayer'
ORDER BY c.ORDINAL_POSITION;
-- Expected: Identical to Step 1b — 12 columns, no changes

-- Step 3c: Confirm all Links are unchanged
SELECT
    t.name AS TableName,
    SUM(p.rows) AS RowCount
FROM sys.tables t
INNER JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN
(0,1)
WHERE t.name LIKE 'LNK_%'
GROUP BY t.name;
-- Expected: All Link row counts identical to Step 1c

-- Step 3d: Confirm the new Satellite exists and is linked to HUB_Taxpayer
SELECT
    t.name AS TableName,
    t.create_date,
    fk.name AS ForeignKeyName,
    OBJECT_NAME(fk.referenced_object_id) AS ReferencedTable
FROM sys.tables t
LEFT JOIN sys.foreign_keys fk ON fk.parent_object_id = t.object_id
WHERE t.name = 'SAT_Taxpayer_Contact';
```

Expected Result:

TableName	create_date	ForeignKeyName	ReferencedTable
SAT_Taxpayer_Contact	(today's date)	FK_SAT_TpContact_HUB	HUB_Taxpayer

Step 4: Load Sample Data into New Satellite

```
-- Step 4: Insert sample contact data (simulating a new SSIS package load)
INSERT INTO SAT_Taxpayer_Contact
    (HUB_Taxpayer_HK, SAT_LoadDate, SAT_RecordSource, SAT_HashDiff,
TaxpayerEmail, TaxpayerPhone)
SELECT
    h.HUB_Taxpayer_HK,
    GETDATE(),
    'TaxSystemDB.ContactModule',
    HASHBYTES('SHA2_256', CONCAT(
        ISNULL('taxpayer' + CAST(ROW_NUMBER() OVER(ORDER BY h.TaxID) AS
VARCHAR) + '@tax.gov.kh', ''), '|',
        ISNULL('+855-' + RIGHT('000000000' + CAST(ABS(CHECKSUM(h.TaxID)) %
1000000000 AS VARCHAR), 9), '')
    )),
    'taxpayer' + CAST(ROW_NUMBER() OVER(ORDER BY h.TaxID) AS VARCHAR) +
'@tax.gov.kh',
    '+855-' + RIGHT('000000000' + CAST(ABS(CHECKSUM(h.TaxID)) % 1000000000 AS
VARCHAR), 9)
FROM DV_Bronze.dbo.HUB_Taxpayer h;

-- Verify
SELECT TOP 5
    CONVERT(VARCHAR(18), HUB_Taxpayer_HK, 1) AS HK_First8,
    TaxpayerEmail, TaxpayerPhone, SAT_LoadDate
FROM SAT_Taxpayer_Contact
ORDER BY SAT_LoadDate;
```

Step 5: Impact Summary

```
-- Final impact assessment: list all Bronze objects and their modification
status
SELECT
    t.name AS ObjectName,
    CASE
        WHEN t.name LIKE 'HUB_%' THEN 'Hub'
        WHEN t.name LIKE 'SAT_%' THEN 'Satellite'
        WHEN t.name LIKE 'LNK_%' THEN 'Link'
    END AS ObjectType,
    CASE
        WHEN t.name = 'SAT_Taxpayer_Contact' THEN '✓ NEW — Created
independently'
        ELSE '✓ UNCHANGED — No modification'
    END AS Impact,
    t.create_date
FROM sys.tables t
WHERE t.name LIKE 'HUB_%' OR t.name LIKE 'SAT_%' OR t.name LIKE 'LNK_%'
```

```sql
ORDER BY
    CASE WHEN t.name LIKE 'HUB_%' THEN 1 WHEN t.name LIKE 'SAT_%' THEN 2 ELSE
3 END,
    t.name;
```

Cleanup (Optional)

```sql
-- Remove the POC table if you want to restore original state
DROP TABLE IF EXISTS DV_Bronze.dbo.SAT_Taxpayer_Contact;
PRINT '✓ POC 1 cleanup complete — SAT_Taxpayer_Contact removed';
```

POC 1 Conclusion: Adding a new attribute required only 1 CREATE TABLE statement. Zero existing Hubs, Satellites, Links, or SSIS packages were modified. This validates the hub-satellite separation pattern [1][5].

POC 2: Full Historical Tracking via Insert-Only Satellite Pattern

Challenge: Traditional update-in-place approaches (SCD Type 1) permanently overwrite previous values, losing the historical audit trail required by government tax regulators.

Objective: Demonstrate that Data Vault 2.0 Satellites preserve all historical versions using the insert-only pattern with HashDiff-based change detection.

Step 1: Verify Current State of Taxpayer ID=1

```sql
-- Connect to: DV_Bronze
USE DV_Bronze;
GO

-- Step 1a: Get the current SAT_Taxpayer record for TaxpayerID 1 (TaxID =
'TAX000001')
SELECT
    CONVERT(VARCHAR(18), sat.HUB_Taxpayer_HK, 1) AS HK_First8,
    sat.LegalBusinessName,
    sat.EstimatedAnnualRevenue,
    sat.SAT_LoadDate,
    sat.SAT_EndDate,
    CASE WHEN sat.SAT_EndDate IS NULL THEN 'Current' ELSE 'Historic' END AS
VersionStatus
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub
    ON sat.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
WHERE hub.TaxID = 'TAX000001'
ORDER BY sat.SAT_LoadDate;
```

Expected Result (after Full Load):

HK_First8	LegalBusinessName	Revenue	SAT_LoadDate	SAT_EndDate	Status
0x3A7F29...	Business 1	500000.00	(load date)	NULL	Current

Only 1 row — no history yet.

Step 2: Simulate First Source Change

```sql
-- Connect to: TaxSystemDB (source system)
USE TaxSystemDB;
GO

-- Step 2a: Update the taxpayer's revenue in the source system
UPDATE Taxpayer
SET EstimatedAnnualRevenue = 350000.00,
    UpdatedDate = GETDATE()
WHERE TaxID = 'TAX000001';

PRINT '✓ Source updated: TAX000001 revenue changed to 350,000';
```

Step 3: Execute Staging + Bronze ETL for Taxpayer

```sql
-- Connect to: DV_Staging
USE DV_Staging;
GO

-- Step 3a: Reload STG_Taxpayer (simulates SSIS Data Flow)
TRUNCATE TABLE STG_Taxpayer;

INSERT INTO STG_Taxpayer
    (TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID,
StructureID,
     RegistrationDate, EstimatedAnnualRevenue, IsActive, RecordSource,
LoadDateTime)
SELECT
    TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID,
StructureID,
    RegistrationDate, EstimatedAnnualRevenue, IsActive,
    'TaxSystemDB' AS RecordSource,
    GETDATE() AS LoadDateTime
FROM TaxSystemDB.dbo.Taxpayer;

PRINT '✓ STG_Taxpayer reloaded: ' + CAST(@@ROWCOUNT AS VARCHAR) + ' rows';
GO

-- Step 3b: Execute Bronze Hub load (idempotent — no new hubs expected)
USE DV_Bronze;
GO
DECLARE @BatchID INT = 999, @RC INT;
EXEC dbo.usp_Load_HUB_Taxpayer @BatchID = @BatchID, @RowCount = @RC OUTPUT;
PRINT '  HUB_Taxpayer loaded: ' + CAST(@RC AS VARCHAR) + ' new rows';

-- Step 3c: Execute Bronze Satellite load (THIS IS WHERE HISTORY IS CAPTURED)
EXEC dbo.usp_Load_SAT_Taxpayer @BatchID = @BatchID, @RowCount = @RC OUTPUT;
PRINT '  SAT_Taxpayer loaded: ' + CAST(@RC AS VARCHAR) + ' changed rows';
```

Step 4: Verify Historical Version Created

```sql
-- Step 4: Query SAT_Taxpayer — should now show 2 versions
SELECT
    CONVERT(VARCHAR(18), sat.HUB_Taxpayer_HK, 1) AS HK_First8,
    sat.LegalBusinessName,
    sat.EstimatedAnnualRevenue,
    sat.SAT_LoadDate,
    sat.SAT_EndDate,
    CASE WHEN sat.SAT_EndDate IS NULL THEN '✓ Current' ELSE '📋 Historic'
END AS VersionStatus
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub
    ON sat.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
WHERE hub.TaxID = 'TAX000001'
ORDER BY sat.SAT_LoadDate;
```

Expected Result:

HK_First8	LegalBusinessName	Revenue	SAT_LoadDate	SAT_EndDate	Status
0x3A7F29...	Business 1	500,000	(original load)	(step 3c time)	📋 Historic
0x3A7F29...	Business 1	350,000	(step 3c time)	NULL	✓ Current

Step 5: Simulate Second Source Change

```sql
-- Step 5a: Second revenue change + business name amendment
USE TaxSystemDB;
GO

UPDATE Taxpayer
SET LegalBusinessName = LegalBusinessName + ' (AMENDED)',
    EstimatedAnnualRevenue = 505000.00,
    UpdatedDate = GETDATE()
WHERE TaxID = 'TAX000001';

PRINT '✓ Source updated: TAX000001 name amended, revenue changed to 505,000';
GO

-- Step 5b: Reload Staging
USE DV_Staging;
GO
TRUNCATE TABLE STG_Taxpayer;
INSERT INTO STG_Taxpayer
    (TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID,
StructureID,
     RegistrationDate, EstimatedAnnualRevenue, IsActive, RecordSource,
LoadDateTime)
SELECT TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID,
StructureID,
```

```
    RegistrationDate, EstimatedAnnualRevenue, IsActive, 'TaxSystemDB',
GETDATE()
FROM TaxSystemDB.dbo.Taxpayer;
GO

-- Step 5c: Execute Bronze loads
USE DV_Bronze;
GO
DECLARE @BatchID INT = 1000, @RC INT;
EXEC dbo.usp_Load_HUB_Taxpayer @BatchID = @BatchID, @RowCount = @RC OUTPUT;
EXEC dbo.usp_Load_SAT_Taxpayer @BatchID = @BatchID, @RowCount = @RC OUTPUT;
PRINT '  SAT_Taxpayer loaded: ' + CAST(@RC AS VARCHAR) + ' changed rows';
```

Step 6: Verify Complete Audit Trail (All 3 Versions)

```
-- Step 6: Final query — all versions preserved
SELECT
    CONVERT(VARCHAR(18), sat.HUB_Taxpayer_HK, 1) AS HK_First8,
    sat.LegalBusinessName,
    sat.EstimatedAnnualRevenue AS Revenue,
    sat.SAT_LoadDate,
    sat.SAT_EndDate,
    CASE WHEN sat.SAT_EndDate IS NULL THEN '✓ V3 (Current)'
         ELSE '📋 V' + CAST(ROW_NUMBER() OVER(ORDER BY sat.SAT_LoadDate) AS
VARCHAR) + ' (Historic)'
    END AS Version
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub
    ON sat.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
WHERE hub.TaxID = 'TAX000001'
ORDER BY sat.SAT_LoadDate;
```

Expected Result:

HK_First8	LegalBusinessName	Revenue	SAT_LoadDate	SAT_EndDate	Version
0x3A7F29...	Business 1	500,000	(original)	(change 1 time)	📋 V1 (Historic)
0x3A7F29...	Business 1	350,000	(change 1 time)	(change 2 time)	📋 V2 (Historic)
0x3A7F29...	Business 1 (AMENDED)	505,000	(change 2 time)	NULL	✓ V3 (Current)

Step 7: Point-in-Time Query (Auditor Use Case)

```
-- Government auditor query: "What was taxpayer TAX000001's revenue as of
[specific date]?"
DECLARE @AuditDate DATETIME = (
    SELECT TOP 1 SAT_LoadDate
    FROM DV_Bronze.dbo.SAT_Taxpayer sat
    INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub ON sat.HUB_Taxpayer_HK =
```

```sql
hub.HUB_Taxpayer_HK
    WHERE hub.TaxID = 'TAX000001'
    ORDER BY SAT_LoadDate  -- Get the first version's date
);

SELECT
    hub.TaxID,
    sat.LegalBusinessName,
    sat.EstimatedAnnualRevenue,
    sat.SAT_LoadDate,
    sat.SAT_EndDate
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub
    ON sat.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
WHERE hub.TaxID = 'TAX000001'
    AND sat.SAT_LoadDate <= @AuditDate
    AND (sat.SAT_EndDate IS NULL OR sat.SAT_EndDate > @AuditDate);
-- Returns: The exact record that was current at @AuditDate
```

Understanding the HashDiff Change Detection Logic

The key mechanism in `usp_Load_SAT_Taxpayer` (from `06_ETL_Bronze_Procedures.sql`):

```sql
-- How the stored procedure works internally:

-- PHASE 1: End-date current records where attributes changed
-- (SAT_HashDiff is SHA-256 hash of all descriptive attributes)
UPDATE sat SET sat.SAT_EndDate = GETDATE()
FROM DV_Bronze.dbo.SAT_Taxpayer sat
INNER JOIN StgHashed src ON sat.HUB_Taxpayer_HK = src.HK
WHERE sat.SAT_EndDate IS NULL           -- Only current records
  AND sat.SAT_HashDiff <> src.HD;       -- Only if attributes actually
changed

-- PHASE 2: Insert new version for changed/new records
INSERT INTO DV_Bronze.dbo.SAT_Taxpayer (...)
SELECT src.HK, GETDATE(), NULL, src.HD, ...
FROM StgHashed src
WHERE NOT EXISTS (
    SELECT 1 FROM DV_Bronze.dbo.SAT_Taxpayer sat
    WHERE sat.HUB_Taxpayer_HK = src.HK
      AND sat.SAT_EndDate IS NULL       -- Current version
      AND sat.SAT_HashDiff = src.HD     -- Same hash = no change = skip
);

-- Result: Only GENUINE changes create new rows
-- Unchanged records are NOT duplicated (HashDiff match → skipped)
```

POC 2 Conclusion: All 3 versions are preserved with precise timestamps. SAT_EndDate = NULL identifies the current version. The insert-only pattern with HASHBYTES('SHA2_256')

ensures only genuine changes are recorded. Government auditors can query any point-in-time.

POC 3: ETL Control Framework with Step-Level Logging and Error Recovery

Challenge: Without proper error handling, ETL failures provide no visibility into what succeeded, what failed, or how to recover — requiring full pipeline restarts.

Objective: Demonstrate the implemented ETL Control Framework's batch-level logging, step-level detail, error capture via SSIS OnError Event Handlers, and selective re-execution of failed steps.

Step 1: Review the ETL Control Tables

```
-- Connect to: ETL_Control
USE ETL_Control;
GO

-- Step 1a: View the 3 core control tables
SELECT 'ETL_BatchLog' AS TableName, COUNT(*) AS RowCount FROM ETL_BatchLog
UNION ALL
SELECT 'ETL_StepLog', COUNT(*) FROM ETL_StepLog
UNION ALL
SELECT 'ETL_ErrorLog', COUNT(*) FROM ETL_ErrorLog;
```

Step 2: Start a New Batch (Simulating Pipeline Execution)

```
-- Step 2a: Start a new batch
DECLARE @BatchID INT;
EXEC usp_StartBatch
    @ProcessName = 'Load_Staging_Full',
    @BatchID = @BatchID OUTPUT;

PRINT '✓ Batch started: BatchID = ' + CAST(@BatchID AS VARCHAR);
-- Save this BatchID for subsequent steps (e.g., @BatchID = 42)
```

Step 3: Execute Steps with Logging (Simulating SSIS Package Execution)

```
-- Use the BatchID from Step 2
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_BatchLog WHERE
BatchStatus = 'Running');
DECLARE @StepID INT, @RC INT;

-- ─── Step 3a: Load_STG_Category (SUCCESS) ───
EXEC usp_StartStep @BatchID = @BatchID, @StepName = 'Load_STG_Category',
@StepID = @StepID OUTPUT;

-- Simulate the actual load
USE DV_Staging;
TRUNCATE TABLE STG_Category;
INSERT INTO STG_Category (CategoryID, CategoryName, CategoryDescription,
```

```sql
IsActive, RecordSource, LoadDateTime)
SELECT CategoryID, CategoryName, CategoryDescription, IsActive,
'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.Category;
SET @RC = @@ROWCOUNT;

USE ETL_Control;
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_Category: ' + CAST(@RC AS VARCHAR) + ' rows';
GO

-- ─── Step 3b: Load_STG_Structure (SUCCESS) ───
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_Control.dbo.ETL_BatchLog
WHERE BatchStatus = 'Running');
DECLARE @StepID INT, @RC INT;

EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_Structure', @StepID = @StepID OUTPUT;
USE DV_Staging;
TRUNCATE TABLE STG_Structure;
INSERT INTO STG_Structure (StructureID, StructureName, StructureDescription,
IsActive, RecordSource, LoadDateTime)
SELECT StructureID, StructureName, StructureDescription, IsActive,
'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.Structure;
SET @RC = @@ROWCOUNT;
USE ETL_Control;
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_Structure: ' + CAST(@RC AS VARCHAR) + ' rows';
GO

-- ─── Step 3c: Load_STG_Taxpayer (SUCCESS) ───
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_Control.dbo.ETL_BatchLog
WHERE BatchStatus = 'Running');
DECLARE @StepID INT, @RC INT;

EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_Taxpayer', @StepID = @StepID OUTPUT;
USE DV_Staging;
TRUNCATE TABLE STG_Taxpayer;
INSERT INTO STG_Taxpayer (TaxpayerID, TaxID, LegalBusinessName, TradingName,
CategoryID, StructureID, RegistrationDate, EstimatedAnnualRevenue, IsActive,
RecordSource, LoadDateTime)
SELECT TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID,
StructureID, RegistrationDate, EstimatedAnnualRevenue, IsActive,
'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.Taxpayer;
SET @RC = @@ROWCOUNT;
USE ETL_Control;
```

```sql
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_Taxpayer: ' + CAST(@RC AS VARCHAR) + ' rows';
GO

-- ─── Step 3d: Load_STG_Payment (SIMULATE FAILURE) ───
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_Control.dbo.ETL_BatchLog
WHERE BatchStatus = 'Running');
DECLARE @StepID INT;

EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_Payment', @StepID = @StepID OUTPUT;

-- Simulate a timeout failure (this is what SSIS OnError Event Handler would
capture)
BEGIN TRY
    RAISERROR('Execution Timeout Expired. The timeout period elapsed prior to
completion of the operation. [Source: TaxSystemDB.dbo.Payment, OLE DB
provider]', 16, 1);
END TRY
BEGIN CATCH
    DECLARE @ErrMsg VARCHAR(MAX) = ERROR_MESSAGE();

    -- Log the step as failed
    USE ETL_Control;
    EXEC usp_EndStep @StepID = @StepID, @Status = 'Failed', @RowCount = 0;

    -- Log detailed error (simulates SSIS OnError Event Handler writing to
ETL_ErrorLog)
    EXEC usp_LogError
        @BatchID = @BatchID,
        @StepLogID = @StepID,
        @ErrorSeverity = 'High',
        @ErrorSource = 'Load_STG_Payment',
        @ErrorMessage = @ErrMsg;

    PRINT '  ✗ Load_STG_Payment: FAILED — ' + @ErrMsg;
END CATCH
GO

-- ─── Step 3e: Load_STG_MonthlyDeclaration (SUCCESS — continues despite
previous failure) ───
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_Control.dbo.ETL_BatchLog
WHERE BatchStatus = 'Running');
DECLARE @StepID INT, @RC INT;

EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_MonthlyDeclaration', @StepID = @StepID OUTPUT;
USE DV_Staging;
TRUNCATE TABLE STG_MonthlyDeclaration;
```

```sql
INSERT INTO STG_MonthlyDeclaration (DeclarationID, TaxpayerID,
DeclarationMonth, DeclarationYear, DueDate, DeclarationDate, GrossRevenue,
TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount,
Status, OfficerID, IsActive, CreatedDate, UpdatedDate, SourceSystem,
RecordSource, LoadDateTime)
SELECT DeclarationID, TaxpayerID, DeclarationMonth, DeclarationYear, DueDate,
DeclarationDate, GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount,
InterestAmount, TotalAmount, Status, OfficerID, IsActive, CreatedDate,
UpdatedDate, SourceSystem, 'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.MonthlyDeclaration;
SET @RC = @@ROWCOUNT;
USE ETL_Control;
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_MonthlyDeclaration: ' + CAST(@RC AS VARCHAR) + ' rows';
GO

-- ─── Step 3f: Load_STG_AnnualDeclaration (SUCCESS) ───
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_Control.dbo.ETL_BatchLog
WHERE BatchStatus = 'Running');
DECLARE @StepID INT, @RC INT;

EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_AnnualDeclaration', @StepID = @StepID OUTPUT;
USE DV_Staging;
TRUNCATE TABLE STG_AnnualDeclaration;
INSERT INTO STG_AnnualDeclaration (AnnualDeclarationID, TaxpayerID,
DeclarationYear, GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount,
InterestAmount, TotalAmount, DeclarationDate, DueDate, Status, OfficerID,
IsActive, CreatedDate, UpdatedDate, SourceSystem, RecordSource, LoadDateTime)
SELECT AnnualDeclarationID, TaxpayerID, DeclarationYear, GrossRevenue,
TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount,
DeclarationDate, DueDate, Status, OfficerID, IsActive, CreatedDate,
UpdatedDate, SourceSystem, 'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.AnnualDeclaration;
SET @RC = @@ROWCOUNT;
USE ETL_Control;
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_AnnualDeclaration: ' + CAST(@RC AS VARCHAR) + ' rows';
GO
```

Step 4: End Batch and Review Results

```sql
USE ETL_Control;
GO

-- Step 4a: End the batch
DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM ETL_BatchLog WHERE
BatchStatus = 'Running');
EXEC usp_EndBatch @BatchID = @BatchID, @Status = 'Partial';
PRINT '✓ Batch ended with status: Partial';
```

```sql
GO

-- Step 4b: View ETL_BatchLog — Pipeline-level status
SELECT
    BatchID, BatchStatus, BatchStartTime, BatchEndTime,
    ExecutionTimeSeconds AS Duration_sec
FROM ETL_BatchLog
WHERE BatchID = (SELECT MAX(BatchID) FROM ETL_BatchLog);
```

Expected Result:

BatchID	BatchStatus	BatchStartTime	BatchEndTime	Duration_sec
(latest)	Partial	(start time)	(end time)	~5-15

```sql
-- Step 4c: View ETL_StepLog — Step-level detail
SELECT
    StepLogID, StepName, StepStatus,
    RecordsProcessed, ExecutionTimeSeconds AS Sec,
    LEFT(ErrorMessage, 80) AS ErrorMessage
FROM ETL_StepLog
WHERE BatchID = (SELECT MAX(BatchID) FROM ETL_BatchLog)
ORDER BY StepLogID;
```

Expected Result:

StepLogID	StepName	StepStatus	RecordsProcessed	Sec	ErrorMessage
...	Load_STG_Category	Success	5	0	
...	Load_STG_Structure	Success	5	0	
...	Load_STG_Taxpayer	Success	1000	1	
...	Load_STG_Payment	**Failed**	0	0	**Execution Timeout Expired...**
...	Load_STG_MonthlyDeclaration	Success	38487	2	
...	Load_STG_AnnualDeclaration	Success	3605	1	

```sql
-- Step 4d: View ETL_ErrorLog — Detailed error capture
SELECT
    ErrorID, BatchID, StepLogID, ErrorSeverity, ErrorSource,
    ErrorMessage, ErrorDateTime
FROM ETL_ErrorLog
WHERE BatchID = (SELECT MAX(BatchID) FROM ETL_BatchLog);
```

Expected Result:

ErrorID	BatchID	StepLogID	Severity	Source	ErrorMessage	ErrorDateTime

ErrorID	BatchID	StepLogID	Severity	Source	ErrorMessage	ErrorDateTime
...	(latest)	(failed step)	High	Load_STG_Payment	Execution Timeout Expired...	(timestamp)

Step 5: Recovery — Re-execute Only Failed Step

```
-- Step 5: The administrator identifies Load_STG_Payment failed and re-runs
ONLY that step
-- No need to re-execute the 5 successful steps

DECLARE @BatchID INT = (SELECT MAX(BatchID) FROM
ETL_Control.dbo.ETL_BatchLog);
DECLARE @StepID INT, @RC INT;

-- Re-execute only the failed step
EXEC ETL_Control.dbo.usp_StartStep @BatchID = @BatchID, @StepName =
'Load_STG_Payment_RETRY', @StepID = @StepID OUTPUT;

USE DV_Staging;
TRUNCATE TABLE STG_Payment;
INSERT INTO STG_Payment (PaymentID, TaxpayerID, DeclarationID,
AnnualDeclarationID, PaymentAmount, PaymentDate, PaymentMethod, Status,
RecordSource, LoadDateTime)
SELECT PaymentID, TaxpayerID, DeclarationID, AnnualDeclarationID,
PaymentAmount, PaymentDate, PaymentMethod, Status, 'TaxSystemDB', GETDATE()
FROM TaxSystemDB.dbo.Payment;
SET @RC = @@ROWCOUNT;

USE ETL_Control;
EXEC usp_EndStep @StepID = @StepID, @Status = 'Success', @RowCount = @RC;
PRINT '  ✓ Load_STG_Payment_RETRY: ' + CAST(@RC AS VARCHAR) + ' rows —
Recovery successful!';

-- Verify: All steps now have data
USE DV_Staging;
SELECT 'STG_Category' AS T, COUNT(*) AS Rows FROM STG_Category
UNION ALL SELECT 'STG_Structure', COUNT(*) FROM STG_Structure
UNION ALL SELECT 'STG_Taxpayer', COUNT(*) FROM STG_Taxpayer
UNION ALL SELECT 'STG_Payment', COUNT(*) FROM STG_Payment
UNION ALL SELECT 'STG_MonthlyDeclaration', COUNT(*) FROM
STG_MonthlyDeclaration
UNION ALL SELECT 'STG_AnnualDeclaration', COUNT(*) FROM
STG_AnnualDeclaration;
```

POC 3 Conclusion: The ETL Control Framework provides: (1) Batch-level status (Partial), (2) Step-level detail identifying Load_STG_Payment as the failed step, (3) Error capture via

OnError Event Handler with detailed error message, (4) Recovery by re-running only the failed step — 7 of 8 steps completed successfully without re-execution.

POC 4: Business Rules Applied via Silver (Business Vault) + Gold (Star Schema)

Challenge: Raw operational data in the Bronze layer contains only individual records with no derived analytics. Tax administrators need compliance scores, payment behavior metrics, and risk classifications that require complex business logic applied consistently.

Objective: Demonstrate how the Silver layer (Business Vault) computes derived metrics from Bronze Hub + Satellite + Link tables, and the Gold layer (Star Schema) organizes these into BI-ready dimensional models.

Step 1: Verify Raw Bronze Data (No Analytics)

```
-- Connect to: DV_Bronze
USE DV_Bronze;
GO

-- Step 1a: Raw Bronze data for TAX000001 — individual records only
SELECT 'Monthly Declarations' AS DataType, COUNT(*) AS Records
FROM LNK_TaxpayerDeclaration lnk
INNER JOIN HUB_Taxpayer hub ON lnk.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
INNER JOIN SAT_MonthlyDecl sat ON lnk.HUB_Declaration_HK =
sat.HUB_Declaration_HK AND sat.SAT_EndDate IS NULL
WHERE hub.TaxID = 'TAX000001'
UNION ALL
SELECT 'Payments', COUNT(*)
FROM LNK_TaxpayerDeclaration lnk_td
INNER JOIN HUB_Taxpayer hub ON lnk_td.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
INNER JOIN LNK_DeclarationPayment lnk_dp ON lnk_td.HUB_Declaration_HK =
lnk_dp.HUB_Declaration_HK
INNER JOIN SAT_Payment sp ON lnk_dp.HUB_Payment_HK = sp.HUB_Payment_HK AND
sp.SAT_EndDate IS NULL
WHERE hub.TaxID = 'TAX000001';
```

Expected Result:

DataType	Records
Monthly Declarations	~36-40
Payments	~20-30

Individual records exist but no derived analytics (compliance score, on-time rates, risk level).

Step 2: Execute Silver Layer — Compute Business Metrics

```
-- Connect to: DV_Silver
USE DV_Silver;
```

```sql
GO

-- Step 2a: Execute BUS_ComplianceScore procedure
DECLARE @BatchID INT = 999, @RC INT;
EXEC dbo.usp_Load_BUS_ComplianceScore
    @BatchID = @BatchID,
    @SnapshotDate = '2026-02-22',
    @RowCount = @RC OUTPUT;
PRINT '✓ BUS_ComplianceScore loaded: ' + CAST(@RC AS VARCHAR) + ' taxpayers
scored';
GO

-- Step 2b: Execute BUS_MonthlyMetrics procedure
DECLARE @BatchID INT = 999, @RC INT;
EXEC dbo.usp_Load_BUS_MonthlyMetrics
    @BatchID = @BatchID,
    @SnapshotDate = '2026-02-22',
    @RowCount = @RC OUTPUT;
PRINT '✓ BUS_MonthlyMetrics loaded: ' + CAST(@RC AS VARCHAR) + ' monthly
records';
```

Step 3: View Derived Compliance Metrics for TAX000001

```sql
-- Step 3: Query BUS_ComplianceScore — derived analytics now available
SELECT
    hub.TaxID,
    sat.LegalBusinessName,
    cs.ComplianceScore,
    cs.FilingOnTimeRate,
    cs.PaymentOnTimeRate,
    cs.PenaltyCount,
    CASE
        WHEN cs.ComplianceScore >= 80 THEN 'LOW'
        WHEN cs.ComplianceScore >= 50 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS RiskClassification
FROM DV_Silver.dbo.BUS_ComplianceScore cs
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub
    ON cs.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
INNER JOIN DV_Bronze.dbo.SAT_Taxpayer sat
    ON hub.HUB_Taxpayer_HK = sat.HUB_Taxpayer_HK AND sat.SAT_EndDate IS NULL
WHERE hub.TaxID = 'TAX000001';
```

Expected Result:

TaxID	LegalBusinessName	ComplianceScore	FilingOnTimeRate	PaymentOnTimeRate	PenaltyCount	Risk
TAX000001	Business 1	~72.5	~78.3	~66.7	2	MEDIUM

The formula: **ComplianceScore = 50% × FilingOnTimeRate + 50% × PaymentOnTimeRate**

Step 4: Execute Gold Layer — Build Star Schema

```
-- Connect to: DV_Gold
USE DV_Gold;
GO

-- Step 4a: Load dimension tables first
DECLARE @BatchID INT = 999, @SD DATETIME = '2026-02-22', @RC INT;

EXEC dbo.usp_Load_DIM_Category @BatchID=@BatchID, @RowCount=@RC OUTPUT;
PRINT '  DIM_Category: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_DIM_Structure @BatchID=@BatchID, @RowCount=@RC OUTPUT;
PRINT '  DIM_Structure: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_DIM_Activity @BatchID=@BatchID, @RowCount=@RC OUTPUT; PRINT
'  DIM_Activity: ' + CAST(@RC AS VARCHAR); EXEC dbo.usp_Load_DIM_Taxpayer
@BatchID=@BatchID, @SnapshotDate=@SD, @RowCount=@RC OUTPUT;
PRINT '  DIM_Taxpayer: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_DIM_Officer @BatchID=@BatchID, @SnapshotDate=@SD,
@RowCount=@RC OUTPUT; PRINT '  DIM_Officer: ' + CAST(@RC AS VARCHAR); EXEC
dbo.usp_Load_DIM_Status @BatchID=@BatchID, @RowCount=@RC OUTPUT;
PRINT '  DIM_Status: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_DIM_PaymentMethod @BatchID=@BatchID, @RowCount=@RC OUTPUT;
PRINT '  DIM_PaymentMethod: ' + CAST(@RC AS VARCHAR);
GO

-- Step 4b: Load fact tables
DECLARE @BatchID INT = 999, @SD DATETIME = '2026-02-22', @RC INT;
EXEC dbo.usp_Load_FACT_MonthlyDeclaration @BatchID=@BatchID,
@SnapshotDate=@SD, @RowCount=@RC OUTPUT;
PRINT '  FACT_MonthlyDeclaration: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_FACT_Payment @BatchID=@BatchID, @SnapshotDate=@SD,
@RowCount=@RC OUTPUT;
PRINT '  FACT_Payment: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_FACT_MonthlySnapshot @BatchID=@BatchID, @SnapshotDate=@SD,
@RowCount=@RC OUTPUT;
PRINT '  FACT_MonthlySnapshot: ' + CAST(@RC AS VARCHAR);
EXEC dbo.usp_Load_FACT_DeclarationLifecycle @BatchID=@BatchID,
@SnapshotDate=@SD, @RowCount=@RC OUTPUT;
PRINT '  FACT_DeclarationLifecycle: ' + CAST(@RC AS VARCHAR);
```

Step 5: Query Gold Star Schema — BI Dashboard Ready

```
-- Step 5a: FACT_MonthlyDeclaration with DIM joins (typical BI dashboard
query)
SELECT
    dt.TaxID AS DIM_Taxpayer,
    ds.StatusCode AS DIM_Status,
    f.DeclarationMonth, f.DeclarationYear,
    f.GrossRevenue, f.TaxAmount, f.TotalAmount,
```

```sql
    -- Join payment data for coverage calculation
    ISNULL(pay.TotalPaid, 0) AS PaymentAmount,
    CASE WHEN f.TotalAmount = 0 THEN 0
         ELSE CAST(ISNULL(pay.TotalPaid, 0) AS DECIMAL(10,2)) / f.TotalAmount
* 100
    END AS CoveragePercent
FROM DV_Gold.dbo.FACT_MonthlyDeclaration f
INNER JOIN DV_Gold.dbo.DIM_Taxpayer dt ON f.DIM_Taxpayer_SK =
dt.DIM_Taxpayer_SK
LEFT JOIN DV_Gold.dbo.DIM_Status ds ON f.DIM_Status_SK = ds.DIM_Status_SK
-- Payment coverage per declaration
OUTER APPLY (
    SELECT SUM(fp.PaymentAmount) AS TotalPaid
    FROM DV_Gold.dbo.FACT_Payment fp
    WHERE fp.DeclarationID = f.DeclarationID
) pay
WHERE dt.TaxID = 'TAX000001' AND dt.IsCurrent = 1
ORDER BY f.DeclarationYear, f.DeclarationMonth;
```

Expected Result (sample rows):

DIM_Taxpayer	DIM_Status	Month	Year	GrossRevenue	TaxAmount	TotalAmount	PaymentAmount	Coverage%
TAX000001	Approved	1	2023	45,000	6,750	6,750	6,750	100%
TAX000001	Approved	2	2023	52,000	7,800	7,800	7,800	100%
TAX000001	Rejected	3	2023	38,000	5,700	5,700	0	0%

```sql
-- Step 5b: Executive dashboard summary — all metrics from Silver + Gold
SELECT
    dt.TaxID,
    dt.LegalBusinessName,
    cs.ComplianceScore,
    cs.FilingOnTimeRate,
    cs.PaymentOnTimeRate,
    cs.PenaltyCount,
    CASE
        WHEN cs.ComplianceScore >= 80 THEN 'LOW'
        WHEN cs.ComplianceScore >= 50 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS RiskLevel,
    COUNT(DISTINCT f.DeclarationID) AS TotalDeclarations,
    SUM(f.GrossRevenue) AS TotalRevenue,
    SUM(f.TaxAmount) AS TotalTax
FROM DV_Gold.dbo.DIM_Taxpayer dt
LEFT JOIN DV_Silver.dbo.BUS_ComplianceScore cs
    ON dt.HUB_Taxpayer_HK = cs.HUB_Taxpayer_HK
```

```sql
LEFT JOIN DV_Gold.dbo.FACT_MonthlyDeclaration f
    ON dt.DIM_Taxpayer_SK = f.DIM_Taxpayer_SK
WHERE dt.TaxID = 'TAX000001' AND dt.IsCurrent = 1
GROUP BY dt.TaxID, dt.LegalBusinessName,
         cs.ComplianceScore, cs.FilingOnTimeRate, cs.PaymentOnTimeRate,
cs.PenaltyCount;
```

POC 4 Conclusion: The Silver layer (Business Vault) computes derived metrics (ComplianceScore = 50% FilingOnTimeRate + 50% PaymentOnTimeRate) using consistent business rules from Bronze Hub + Satellite + Link tables. The Gold layer (Star Schema) organizes these into DIM/FACT joins for BI dashboard consumption. Tax administrators get instant, consistent analytics without writing ad-hoc queries across raw Bronze tables.

Summary of POC Results

#	Challenge	DV 2.0 Solution	POC Result	Script Reference
1	Schema Rigidity	Hub-Satellite separation: CREATE new Satellite	Zero existing objects modified	04_DDL_Architecture_DW.sql
2	Historical Tracking	Insert-only Satellite with HashDiff	All versions preserved with timestamps	06_ETL_Bronze_Procedures.sql
3	Scalable ETL	BatchLog + StepLog + OnError Event Handlers	Step-level visibility with selective recovery	03_ETL_Control_Setup.sql
4	Business Rules Gap	Silver (Business Vault) + Gold (Star Schema)	Pre-computed scores, trends, risk levels	07_ETL_Silver + 08_ETL_Gold

---

Script Cross-Reference

POC	Primary Script(s)	Key Procedure/Object
POC 1	04_DDL_Architecture_DW.sql	HUB_Taxpayer, SAT_Taxpayer DDL
POC 2	06_ETL_Bronze_Procedures.sql	usp_Load_SAT_Taxpayer (HashDiff logic)
POC 2	12_IncrementalTest_SourceChanges.sql	UPDATE Taxpayer SET … (source changes)
POC 3	03_ETL_Control_Setup.sql	ETL_BatchLog, ETL_StepLog, ETL_ErrorLog
POC 3	03_ETL_Control_Setup.sql	usp_StartBatch, usp_EndBatch, usp_StartStep, usp_EndStep, usp_LogError

POC	Primary Script(s)	Key Procedure/Object
POC 4	07_ETL_Silver_Procedures.sql	usp_Load_BUS_ComplianceScore
POC 4	08_ETL_Gold_Procedures.sql	usp_Load_FACT_MonthlyDeclaration, usp_Load_DIM_Taxpayer

End of POC Implementation Guide